ARCH TECHNOLOGIES
Data Science **Internship Report**

Full Name: *Arish Rafiq*
Internship Domain: Data Science
Email Address: *arish.rafiq03@gmail.com*
Phone Number: *+92 3182469992*

Table of Contents

1. Introduction
2. Project 1: Titanic Survival Prediction
3. Project 2: NVIDIA Stock Price Prediction
4. Tools & Technologies
5. Conclusion
6. Code Section

1. Introduction

This report presents two data science projects completed during the internship:

- Titanic Survival Prediction using Machine Learning
- NVIDIA Stock Price Prediction using Linear Regression

The objective was to apply data preprocessing, visualization, and predictive modeling techniques on real-world datasets.

Project 1: Titanic Survival Prediction

2.1 Objective

To predict whether a passenger survived the Titanic disaster using classification algorithms.

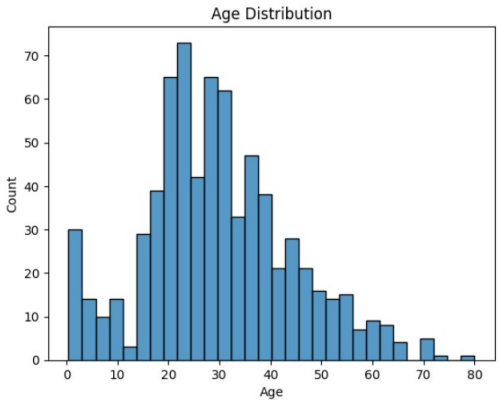
2.2 Dataset

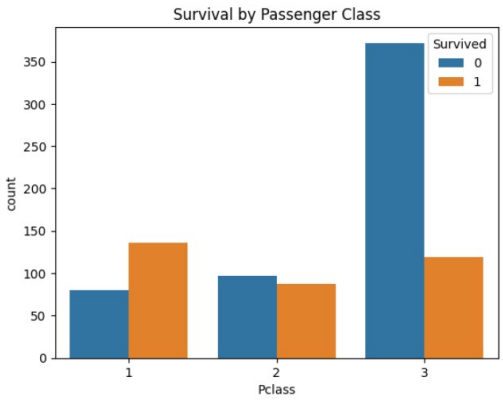
- Dataset: Titanic Dataset
- Features: Age, Sex, Fare, Pclass, Embarked, etc.
- Target Variable: Survived (0 = No, 1 = Yes)
- Link: Titanic Dataset

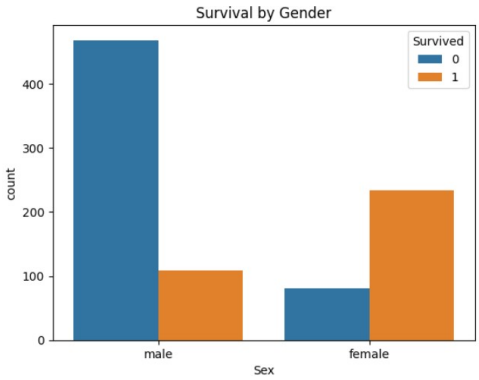
2.3 Data Preprocessing

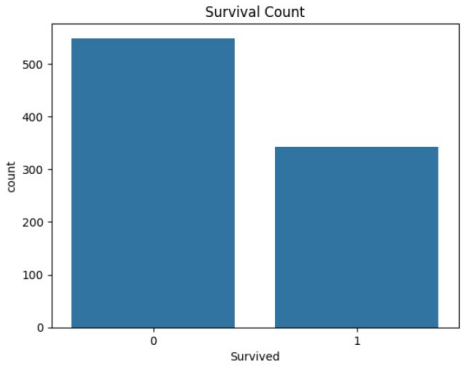
- Handled missing values:
  - Age → Median
  - Embarked → Mode
  - Fare → Median
- Dropped unnecessary columns:
  - Cabin, Name, Ticket, PassengerId
- Encoded categorical variables:
  - Sex, Embarked using LabelEncoder

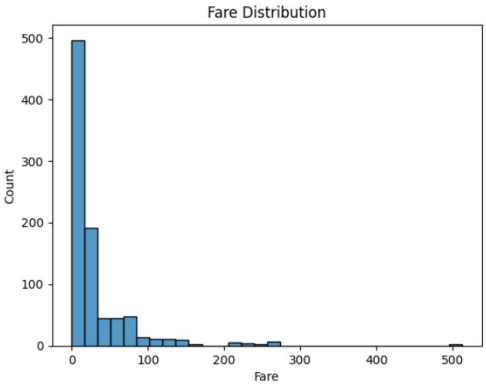
2.4 Data Visualization

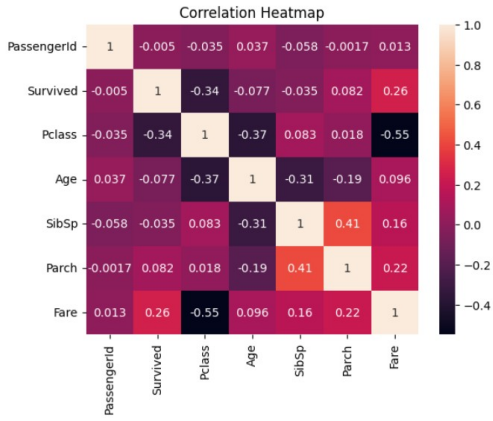












2.5 Model Implementation

- Algorithm Used: Random Forest Classifier
- Train-Test Split: 80% training, 20% testing

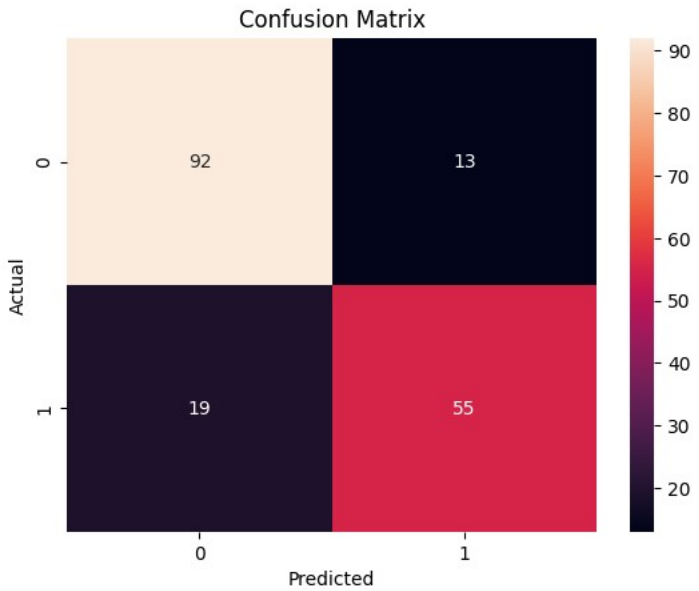
```
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

2.6 Model Evaluation

Model performance was evaluated using:

- **Accuracy Score:** Overall correctness
- **Classification Report:** Precision, Recall, F1-score
- **Confusion Matrix:** Detailed prediction breakdown

```
Model Evaluation:

Accuracy:
0.8212290502793296

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.88      0.85       105
           1       0.81      0.74      0.77        74

    accuracy                           0.82       179
   macro avg       0.82      0.81      0.81       179
weighted avg       0.82      0.82      0.82       179


Confusion Matrix:
[[92 13]
 [19 55]]
```



2.7 Result

The Random Forest model achieved good accuracy and effectively classified survival outcomes based on passenger features.

3. Project 2: NVIDIA Stock Price Prediction

3.1 Objective

To analyze stock price trends and predict closing prices using regression techniques.

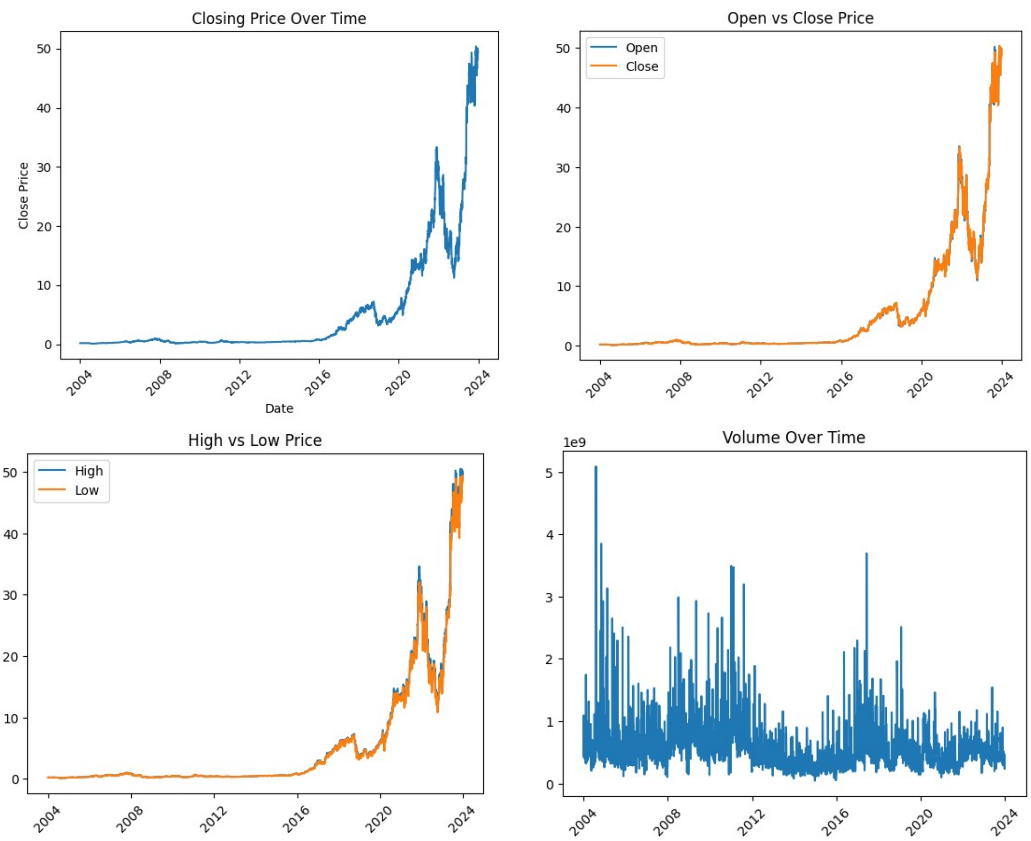
3.2 Dataset

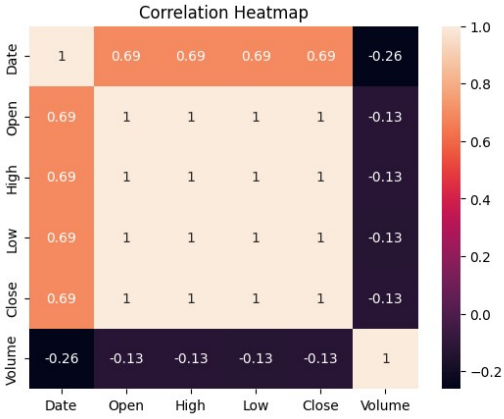
- Dataset: NVIDIA Stock Prices
- Features: Open, High, Low, Close, Volume
- Target Variable: Close Price
- Link: Nvidia Daily Stock Price Data

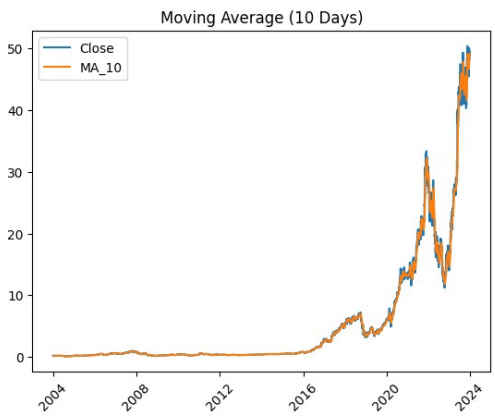
3.3 Data Preprocessing

- Handled missing values using forward fill
- Converted Date column to datetime
- Sorted data by date

3.4 Data Visualization







3.5 Model Implementation

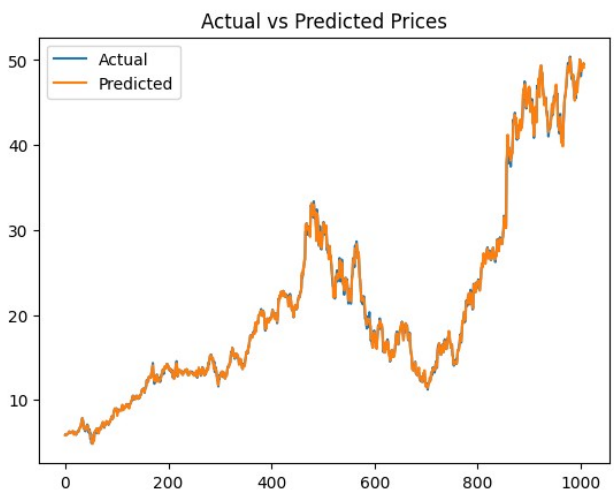
- Algorithm Used: Linear Regression
- Data Split: 80% training, 20% testing (time-based split)

```
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

3.6 Model Evaluation

- Mean Squared Error (MSE)
- R^2 Score

```
Model Evaluation:

Mean Squared Error (MSE):
0.06628604304882971

R2 Score:
0.9995201045282917
```



3.7 Result

The Linear Regression model successfully predicted stock closing prices with reasonable accuracy, showing trends aligned with actual values.

4. Tools & Technologies

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn

5. Conclusion

These projects demonstrate practical implementation of:

- Data cleaning and preprocessing
- Exploratory Data Analysis (EDA)
- Machine Learning models
- Model evaluation techniques

The internship provided hands-on experience in solving real-world data problems.

Code Section:

Titanic Survival Prediction

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

df = pd.read_csv("Titanic-Dataset.csv")

print("First 5 Rows:")
print(df.head())

print("\nDataset Info:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

#Survival Count
plt.figure()
sns.countplot(x='Survived', data=df)
plt.title("Survival Count")
plt.show()

#Survival by Gender
plt.figure()
sns.countplot(x='Sex', hue='Survived', data=df)
plt.title("Survival by Gender")
plt.show()

#Survival by Class
plt.figure()
sns.countplot(x='Pclass', hue='Survived', data=df)
plt.title("Survival by Passenger Class")
plt.show()
```

```python
#Age Distribution
plt.figure()
sns.histplot(df['Age'], bins=30)
plt.title("Age Distribution")
plt.show()

#Fare Distribution
plt.figure()
sns.histplot(df['Fare'], bins=30)
plt.title("Fare Distribution")
plt.show()

#Correlation Heatmap
plt.figure()
sns.heatmap(df.corr(numeric_only=True), annot=True)
plt.title("Correlation Heatmap")
plt.show()

# Handle missing values
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
df['Fare'].fillna(df['Fare'].median(), inplace=True)

# Drop unnecessary columns
df.drop(columns=['Cabin', 'Name', 'Ticket', 'PassengerId'], inplace=True)

# Encode categorical data
le = LabelEncoder()
df['Sex'] = le.fit_transform(df['Sex'])
df['Embarked'] = le.fit_transform(df['Embarked'])

X = df.drop('Survived', axis=1)
y = df['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```python
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\n Model Evaluation:")

print("\nAccuracy:")
print(accuracy_score(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# CONFUSION MATRIX VISUAL
plt.figure()
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

NVIDIA Stock Price Prediction

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv("nvidia_stock_prices.csv")

print("First 5 Rows:")
print(df.head())

print("\nDataset Info:")
print(df.info())

print("\nStatistical Summary:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

df.ffill(inplace=True)

# Convert Date & Sort
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'])
    df = df.sort_values('Date')


# Closing Price Trend
plt.figure()
plt.plot(df['Date'], df['Close'])
plt.title("Closing Price Over Time")
plt.xlabel("Date")
plt.ylabel("Close Price")
plt.xticks(rotation=45)
plt.show()
```

```python
# Open vs Close
plt.figure()
plt.plot(df['Date'], df['Open'], label='Open')
plt.plot(df['Date'], df['Close'], label='Close')
plt.legend()
plt.title("Open vs Close Price")
plt.xticks(rotation=45)
plt.show()

# High vs Low
plt.figure()
plt.plot(df['Date'], df['High'], label='High')
plt.plot(df['Date'], df['Low'], label='Low')
plt.legend()
plt.title("High vs Low Price")
plt.xticks(rotation=45)
plt.show()

# Volume Trend
plt.figure()
plt.plot(df['Date'], df['Volume'])
plt.title("Volume Over Time")
plt.xticks(rotation=45)
plt.show()

# Correlation Heatmap
plt.figure()
sns.heatmap(df.corr(), annot=True)
plt.title("Correlation Heatmap")
plt.show()

# Moving Average
df['MA_10'] = df['Close'].rolling(window=10).mean()

plt.figure()
plt.plot(df['Date'], df['Close'], label='Close')
plt.plot(df['Date'], df['MA_10'], label='MA_10')
plt.legend()
plt.title("Moving Average (10 Days)")
plt.xticks(rotation=45)
plt.show()
```

```python
features = ['Open', 'High', 'Low', 'Volume']
X = df[features]
y = df['Close']


split_index = int(len(df) * 0.8)

X_train = X[:split_index]
X_test = X[split_index:]

y_train = y[:split_index]
y_test = y[split_index:]

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)


print("\n Model Evaluation:")

print("\nMean Squared Error (MSE):")
print(mean_squared_error(y_test, y_pred))

print("\nR2 Score:")
print(r2_score(y_test, y_pred))

plt.figure()
plt.plot(y_test.values, label='Actual')
plt.plot(y_pred, label='Predicted')
plt.legend()
plt.title("Actual vs Predicted Prices")
plt.show()
```